

Упражнение 1

това е първият коментар

SPAM = 1 # а това е вторият коментар # ... а ето и трети!

STRING = "# Това не е коментар."

1. Използване на Python като калкулатор

Нека изпробваме няколко прости команди на Питон. Стартирайте интерпретатора и изчакайте появяването на първичния промпт, '>>>'.

1.1. Числа

Примери:

```
>>> 2+2
4
>>> # Това е коментар
... 2+2
4
>>> 2+2 # и коментар на същия ред заедно с код
4
>>> (50-5*6)/4
5
>>> # Целочисленото деление връща цялата част:
... 7/3
2
>>> 7/-3
-3
```

Както в C, знакът за равенство ('=') се използва за присвояване на стойност на променлива. Стойността на присвояването не се изписва:

```
>>> width = 20
>>> height = 5*9
>>> width * height
900
```

Една стойност може да бъде присвоена едновременно на няколко променливи:

```
>>> x = y = z = 0 # Нулираме x, y и z
>>> x
0
>>> y
0
>>> z
0
```

Съществува пълна поддръжка на плаваща запетая; операторите със смесен тип на операндите превръщат целочисления си операнд в тип с плаваща запетая:

```
>>> 4 * 2.5 / 3.3
3.0303030303030303
>>> 7.0 / 2
3.5
```

Поддържат се също и комплексни числа; имагинерните числа се записват със суфикс 'j' или 'J'. Комплексните числа с ненулев реален компонент се записват като '(реал+имагj)', или могат да бъдат създадени с функцията 'complex(реал, имаг)'.

```
>>> 1j * 1J (-1+0j)
-1j
>>> 1j * complex(0,1) (-1+0j)
-1j
>>> 3+1j*3 (3+3j)
(3+3j)
>>> (3+1j)*3 (9+3j)
(9+3j)
```

```
>>> (1+2j)/(1+1j) (1.5+0.5j)
```

Комплексните числа са представени винаги като две числа с плаваща запетая, реална и имагинерна част. За да извлечете тези две части от комплексното число `z`, използвайте `z.real` и `z.imag`.

```
>>> a=1.5+0.5j
>>> a.real 1.5
>>> a.imag 0.5
```

Преобразуващите функции към плаваща запетая и целочислен тип (`float()`, `int()` и `long()`) не работят с комплексни числа — няма правилен начин да се преобразува комплексно число в реално. Използвайте `abs(z)` за да вземете величината му (като плаваща запетая) или `z.real`, за да вземете реалната му част.

```
>>> a=1.5+0.5j
>>> float(a)
Traceback (innermost last): File "<stdin>", line 1, in ?
TypeError: can't convert complex to float; use e.g. abs(z)
>>> a.real 1.5
>>> abs(a) 1.5811388300841898
```

В интерактивен режим, последният изписан израз е присвоен на променливата `_`. Това означава, че когато използвате Питон като настолен калкулатор, ще е малко по-лесно да продължите изчисленията, например:

```
>>> tax = 17.5 / 100
>>> price = 3.50
>>> price * tax 0.61249999999999993
>>> price + _ 4.1124999999999998
>>> round(_, 2)
4.1100000000000003
```

Потребителят трябва да разглежда тази променлива като “само за четене”. Не и присвоявайте стойност явно — тогава ще създадете независима локална променлива със същото име, скриваща вградената променлива с магическо поведение.

1.2. Символни низове

Освен с числа, Питон може да борава и със символни низове, които могат да бъдат изразени по няколко начина. Те могат да са затворени в единични или двойни кавички:

```
>>> 'spam eggs' 'spam eggs'
>>> 'doesn\'t' "doesn't"
>>> "doesn't" "doesn't"
>>> '"Yes," he said.'
'"Yes," he said.'
>>> "\"Yes,\" he said.\" \"Yes,\" he said.'
>>> '"Isn\'t," she said.'
'"Isn\'t," she said.'
```

Символните низове могат да се прострат на повече редове по няколко начина. Новите редове могат да се избегнат с обратни наклонени черти, т.е.:

```
hello = "Това е доста дълъг символен низ, съдържащ\n\n няколко реда текст така както бихте го\nнаправили в C.\n\n"
```

Забележете, че празното пространство в началото на реда\ има значение.\n"

```
print hello
```

Което ще отпечата следното:

Това е доста дълъг символен низ, съдържащ
няколко реда текст така както бихте го направили в C.
Забележете, че празното пространство в началото на реда има значение.

Или пък, символните низове могат да бъдат затворени в тройни кавички: `"""` или `'''`.
Когато използвате тройните кавички, няма нужда да избягвате края на редовете, но те
ще бъдат включени в символния низ.

```
print """
Usage: thingy [OPTIONS]
-h          Display this usage message
-H hostname  Hostname to connect to
"""
```

предизвиква следния изход:

```
Usage: thingy [OPTIONS]
-h          Display this usage message
-H hostname  Hostname to connect to
```

Интерпретаторът извежда резултата от операциите със символни низове по същия начин, по който те са въведени — в кавички, а кавичките и другите забавни знаци са предшествани от обратна наклонена черта, за да се покаже точната им стойност. Символният низ е затворен в двойни кавички, ако съдържа единична кавичка и не съдържа никакви двойни кавички. В противен случай символният низ е затворен в единични кавички. (Операторът `print`, описан по-долу, може да бъде използван за отпечатване на символни низове без кавички и обратно наклонени черти, предшестващи специалните знаци.)

Символните низове могат да бъдат сливани (слепвани заедно) чрез оператора `+`, и повтаряни с `*`:

```
>>> word = 'Help' + 'A'
>>> word
'HelpA'
>>> '<' + word*5 + '>'
'<HelpAHelpAHelpAHelpAHelpA>'
```

Два символни низа един до друг се събират автоматично: първия ред по-горе може да бъде написан като `'word = 'Help' 'A''`; това работи само при два символни низа, но не и при произволни изрази със символни низове.

```
>>> import string
>>> 'str' 'ing'          #      <-    Това е      наред 'string'
>>> string.strip('str') + 'ing'  #      <-    Това е      наред 'string'
>>> string.strip('str') 'ing'    #      <-    Това е      невалидно
File "<stdin>", line 1 string.strip('str') 'ing'
^
SyntaxError: invalid syntax
```

Символните низове могат да бъдат индексирани; както в C, първият знак в един символен низ има индекс 0. Не съществува отделен тип данни за самостоятелен знак (`char`); знакът просто е символен низ с дължина едно. Подобно на езика Icon, подстринговете могат да бъдат определяни чрез конвенцията за изрязване — два индекса разделени с двоеточие.

```
>>> word[4]
'A'
```

```
>>> word[0:2] 'He'
>>> word[2:4] 'lp'
```

За разлика от C, символните низове в Питон не могат да бъдат променяни. Опитът за присвояване на индексирана позиция в символен низ предизвиква грешка:

```
>>> word[0] = 'x' Traceback (innermost last):
File "<stdin>", line 1, in ?
TypeError: object doesn't support item assignment
>>> word[-1] = 'Splat' Traceback (innermost last):
File "<stdin>", line 1, in ?
TypeError: object doesn't support slice assignment
```

Обаче лесно и ефикасно можете да създадете нов символен низ, съдържащ желаното съчетание:

```
>>> 'x' + word[1:] # Всичко освен първият знак
'xelpA'
>>> 'Splat' + word[-1:] # Всичко освен последния знак (за последен се счита A и # се брои от
дясно на ляво затова има отрицателна стойност)
'SplatA'
```

При изрязването индексите имат полезни стойности по подразбиране; изпуснат първи индекс се приема за нула, а изпуснат втори индекс приема стойност, равна на размера на символния низ, който се изрязва.

```
>>> word[:2] # Първите два знака 'He'
>>> word[2:] # Всичко освен първите два знака 'lpA'
```

Ето и едно полезно свойство на изрязващите операции: `s[:i] + s[i:]` е равно на `s`.

```
>>> word[:2] + word[2:] 'HelpA'
>>> word[:3] + word[3:] 'HelpA'
```

Индексите за изрязване се обработват по следният начин: индекс, който е твърде голям, се заменя с размера на символния низ, както и ако горната граница на идекса е по-малка от долната, се връща празен символен низ.

```
>>> word[1:100]
'elpA'
>>> word[10:] ''
>>> word[2:1] ''
```

Индексите могат да имат отрицателни стойности, за да започнат броенето от дясно на ляво.

Например:

```
>>> word[-1] # Последният знак 'A'
>>> word[-2] # Предпоследният знак 'p'
>>> word[-2:] # Последните два знака 'pA'
>>> word[:-2] # Всичко освен последните два знака 'Hel'
```

Забележете обаче, че `-0` наистина е същото като `0`, така че не се брои от дясно на ляво!

```
>>> word[-0] # (тъй като -0 е равно на 0) 'H'
```

Негативните индекси за изрязване, излизащи извън рамките на символния низ, се окастрят. Но не опитвайте това с индексите за единичен елемент (т.е. когато не изрязвате).

```
>>> word[-100:]
```

```
'HelpA'
>>> word[-10] # грешка Traceback (innermost last):
File "<stdin>", line 1
IndexError: string index out of range
```

Най-добрият начин да запомните как работи изрязването е като мислите за индексите като за сочеци между знаците, като левия край на първия знак е номериран с 0. Тогава десният край на последния знак от символен низ, състоящ се от n знака има индекс n, например:

```
+---+---+---+---+
| H | e | l | p | A |
+---+---+---+---+
0  1  2  3  4      5
-5 -4 -3 -2 -1
```

Първият ред от числа дава позицията на индексите 0...5 в символния низ; вторият ред дава съответните им отрицателни индекси. Изрезката от i до j съдържа всички знаци между краищата, отбелязани с i и j, респективно.

За неотрицателните индекси дължината на изрезката е разликата между индексите, ако и двата са в рамките на символния низ. Сиреч, дължината на word[1:3] е 2.

Вградената функция len() връща дължината на даден символен низ:

```
>>> s = 'непротивоконституционствувателствувайте'
>>> len(s) 39
```

1.3. Символни низове Уникод

В Питон 2.0 програмистът притежава един нов тип данни за съхраняване на текст – обектът Уникод (Unicode). Той може да бъде използван за съхраняване и обработка на Уникод данни (виж <http://www.unicode.org>) и се интегрира добре със досега съществуващите обекти от тип сим-волен низ, като при необходимост се извършва автоматично преобразуване между двата типа.

Предимството на Уникод е, че предоставя отделно число за всеки отделен символ от всички азбуки, използвани в съвременните и древните текстове. Преди това имаше само 256 възможни числа за символите на една азбука и текстовете обикновено бяха “обвързани” към определена кодова страница, която дефинираше съответствието между числата и символите от азбуката. Това доведе до много голямо объркване, особено по отношение на интернационализацията (обикновено се пише като ‘i18n’).

В Питон, създаването на Уникод низове е също толкова просто, колкото и създаването на обикновени символни низове:

```
>>> u'Hello World !' u'Hello World !'
```

Малката буква ‘u’ в пред кавичката показва, че трябва да се създаде Уникод низ. Ако искате да включите специални символи в низа, можете да го направите чрез кодирането Unicode-Escape. Как става това се вижда от следващия пример:

```
>>> u'Hello\\u0020World !' u'Hello World !'
```

Заобикалящата последователност nnu0020 означава, че на дадената позиция трябва да се вмъкне Уникод символ с шестнадесетично число 0x0020 (знак за интервал).

Другите символи се интерпретират чрез съответната им числова стойност така, сякаш са Уникод числа. Процесът на въвеждане на Уникод е значително опростен поради това, че първите 256 Уникод символа съвпадат със стандартното кодиране Latin-1, използвано в много западни страни.

За специалистите съществува още един суров режим, точно както и при нормалните символни низове. Ако в началото на низа поставите малката буква ‘r’, Питон ще използва

кодирането Raw-Unicode-Escape. То ще приложи горното превръщане на `nnuXXXX` само ако намери нечетен брой обратно наклонени черти преди малката буква 'u'.

```
>>> ur'Hello\u0020World !' u'Hello World !'
>>> ur'Hello\\u0020World !' u'Hello\\\\u0020World !'
```

Освен стандартните кодирания, Питон предлага и голям набор от други начини за създаване на Уникод низове, основани на познати кодирания.

Вградената функция `unicode()` предоставя достъп до всички официално регистрирани кодеци (КОДЕК означава КОдер и ДЕКОдер) в организацията на Уникод. Някои от най-известните кодира-ния, които тези кодеци могат да превръщат са Latin-1, ASCII, UTF-8 и UTF-16.5 Последните две са кодирания с променлив размер, позволяващи съхраняване на Уникод символи в 8 или 16 бита. Питон по подразбиране използва кодирането UTF-8. Това се забелязва когато Уникод низовете се извеждат в терминала или се записват във файл.

Ако разполагате с данни в някакво конкретно кодиране и искате да получите съответния Уникод низ, можете да използвате вградената функция `unicode()` с името на кодирането като втори аргумент.

```
>>> p = unicode('\320\274\320\270\321\200', 'UTF-8')
>>> p
u'\u043C\u0438\u0440'
```

За да превърнете същия Уникод низ обратно в обикновен низ с първоначалното му кодиране, Уникод обекта предлага метода `encode()`.

```
>>> p.encode('UTF-8') '\320\274\320\270\321\200'
```

3.2. Първи стъпки в програмирането

Разбира се, можем да използваме Питон за по-сложни задачи от събиране на две и две. Например можем да изведем началната поредица от числата на Фибоначи ето така:

```
>>> # Числа на Фибоначи:
... # сумата на два елемента определя следващия
... a, b = 0, 1
>>> while b < 10:
...     print b
...     a, b = b, a+b
...
...
...
1
1
2
3
5
8
```

Този пример въвежда няколко нови свойства.

– Първият ред съдържа многократно присвояване: променливите `a` и `b` едновременно получават новите стойности 0 и 1. В последния ред това е използвано отново, показвайки че изразите от дясната страна се изчисляват преди каквото и да било присвояване. Изразите от дясната страна се изчисляват от ляво на дясно.

– Цикълът `while` се изпълнява дотогава, докато условието (в случая: `b < 10`) е вярно. В Питон, подобно на С, всяка ненулева стойност е истина; нула е неистина. Също така, условието може да бъде символен низ или списък, всъщност каквато и да е редица (sequence). Всичко с ненулева дължина е истина, празните редици са неистина. Проверката, използвана в примера, е просто сравнение. Стандартните оператори за сравнение се изписват по същия начин, както и в С: `<` (по-малко), `>` (по-голямо), `==` (равно), `<=` (по-малко или равно), `>=` (по-голямо или равно) и `!=` (неравно).

– Тялото на цикъла е с отстъп: отстъпът е начина на Питон за групиране на операторите. Питон не предлага (засега!) интелигентно средство за редактиране на редовете, така че ще трябва да въвеждате табулация или интервал(и) за всеки ред с отстъп. В практиката ще подготвяте по-сложния вход за Питон с текстов редактор; повечето текстови редактори имат средство за автоматичен отстъп. Когато съставен оператор се въвежда интерактивно, той трябва да бъде последван от празен ред за да се укаже, че е приключил (тъй като граматичният анализатор не може да познае кога сте написали последния ред). Обърнете внимание, че всички редове, образуващи общ блок, трябва да бъдат с еднакъв отстъп.

– Операторът `print` изписва стойността на израза (или изразите), които са му подадени. Той се различава от простото изписване на израза който искате да покажете (както правихме по-горе в примерите с калкулатора) по начина по който се обработват символните низовеб и съставните изрази. Символните низове се отпечатват без кавички и между елементите се вмъква интервал, така че можете добре да форматирате нещата. Например:

```
>>> i = 256*256
>>> print 'Стойността на i е', i
Стойността на i е
65536
```

Чрез оставена накрая запетайка се избягва преминаването на нов ред след изхода.

```
>>> a, b = 0, 1
>>> while b < 1000:
...     print b,
...     a, b = b, a+b
...
1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
```

Забележете, че интерпретаторът вмъква нов ред преди да е показал следващия си промпт, ако последният ред не е бил завършен.

Повече средства за управление на изпълнението

Освен току-що представения оператор `while`, Питон познава и обичайните оператори за управление на изпълнението, познати от други езици, но с някои особености.

4.1. if-конструкции

Може би най-познатият вид конструкции са `if`-конструкциите. Например:

```
>>> # x = int(raw_input("Моля, въведете едно число: "))
>>> if x < 0:
...     x = 0
...     print 'Отрицателно, сменено на нула'
... elif x == 0:
...     print 'Нула'
... elif x == 1:
...     print 'Едно'
... else:
...     print 'Повече'
...
```

Може да има нула или повече `elif` части. Частта `else` е незадължителна. Ключовата дума `'elif'` е съкратено от `'else if'`, и е полезна за избягване на излишния отстъп. Поредицата `if . . . elif . . . elif . . .` замества конструкциите `switch` или `case`, които намираме в други езици.

4.2. **for-конструкции**

Операторът `for` в Питон малко се различава от това, което може би сте използвали в С или Паскал. Вместо винаги да се итерира върху някаква аритметична прогресия на числа (като в Паскал), или да се дава на потребителя възможност да определя и стъпката на итерация, и условието за спиране (като в С), в Питон операторът `for` итерира върху елементите на каквато и да е редица (напр. списък или символен низ), в реда, по който те се явяват в редицата. Например (без намерение за каламбур):

```
>>> # Да измерим няколко символни низа:
... a = ['котка', 'прозорец', 'изхвърляне навън']
>>> for x in a:
...     print x, len(x)
...
котка 5
прозорец 8
изхвърляне навън 16
```

Не е безопасно да се променя редицата, докато се итерира върху нея в цикъл (това може да се случи само с променливите типове редици, т.е. списъците). Ако трябва да промените списъка върху който итерирате, напр. за да дублирате избрани елементи, тогава е необходимо да итерирате върху копие. Конвенцията за изрязване улеснява това частично:

```
>>> for x in a[:]: # прави копие чрез изрязване на целия списък
...     if len(x) > 8: a.insert(0, x)
...
>>> a
['изхвърляне навън', 'котка', 'прозорец', 'изхвърляне навън']
```

4.3. **Функцията range()**

Ако трябва да итерирате върху редица от числа, тогава идва на помощ вградената функция `range()`. Тя генерира списъци, съдържащи аритметични прогресии, напр.:

```
>>> range(10)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Подадената крайна точка никога не е част от генерирания списък; `range(10)` генерира списък от 10 стойности, съвпадащи с легалните индекси за елементите на редица с дължина 10. Възможно е да накарате диапазона да започва от друго число, или да определите различна стъпка на увеличение (дори отрицателна):

```
>>> range(5, 10)
[5, 6, 7, 8, 9]
>>> range(0, 10, 3)
[0, 3, 6, 9]
>>> range(-10, -100, -30)
[-10, -40, -70]
```

За да итерирате върху индексите в редица, комбинирайте функциите `range()` и `len()` както следва:

```
>>> a = ['Когато', 'бях', 'овчарче', 'и', 'овците', 'пасях']
>>> for i in range(len(a)):
...     print i, a[i]
...
0 Когато
1 бях
2 овчарче
3 и
4 овците
5 пасях
```

4.4. Оператори break и continue, и клаузи else в циклите

Операторът break, както в C, излиза от най-вътрешния for или while цикъл.

Операторът continue, също взет назаем от C, продължава със следващата итерация в цикъла.

Циклите могат да имат клауза else; тя се изпълнява когато цикълът завърши чрез изчерпване на списъка (при for) или когато условието стане неистина (при while), но не и когато цикълът за-върши с оператор break. Това е демонстрирано от следващия цикъл, който търси прости (неделими) числа:

```
>>> for n in range(2, 10):
...     for x in range(2, n):
...         if n % x == 0:
...             print n, 'е равно на', x, '*', n/x
...             break
...         else:
...             print n, 'е просто число'
...
...
2 е просто число
3 е просто число
4 е равно на 2 * 2
5 е просто число
6 е равно на 2 * 3
7 е просто число
8 е равно на 2 * 4
9 е равно на 3 * 3
```

4.5. Операторът pass

Операторът pass не прави нищо. Той може да бъде използван, когато синтактически се очаква оператор, но програмата изисква бездействие. Например:

```
>>> while 1:
...     pass # Чакай за прекъсване от клавиатурата
...
```